CSC315 Number System, Codes and Addressing Modes (2)

Condensed Study Notes
Generated: 2026-05-24 23:18 UTC

COMPUTER ARCHITECTURE AND ORGANIZATION AFIT, KADUNA LECTURE NOTE

This lecture note introduces the fundamental concepts of Computer Architecture and Organization.

Computer Architecture refers to the attributes of a system visible to a programmer. These attributes have a direct impact on the logical execution of a program. Think of it like the blueprint of a house – it shows where the rooms are, how they connect, and what the overall structure is, influencing how you would navigate and use the house.

Computer Organization, on the other hand, deals with the operational units and their interconnections that realize the architectural specifications. This is like the actual construction of the house – the plumbing, electrical wiring, and structural beams that make the blueprint a reality. It's about how the components are physically put together and work together.

In essence:

- Architecture is about *what* the system does from a programmer's perspective.
- Organization is about *how* it does it.

1.0 Number System

Different number systems are used in computing, each with its own set of symbols and a base (or radix). The base determines the range of numbers available in that system.

The most commonly used number systems are:

- **Decimal Number System**: This is the system we use every day. It has a base of 10, meaning it uses 10 digits (0 through 9).
- **Binary Number System**: This system is fundamental to computers. It has a base of 2, meaning it only uses two digits: 0 and 1.
- **Octal Number System**: This system has a base of 8, using digits from 0 through 7.
- **Hexadecimal Number System**: This system has a base of 16, using digits 0 through 9 and letters A through F to represent values 10 through 15.

Base (or Radix): In any number system, the base 'r' defines the number of unique digits available. These digits range from 0 up to r-1. For example, a system with base 'r' will have 'r' total distinct numbers. We can create various number systems by choosing a base 'r' that is greater than or equal to two.

1.1 Decimal Number System

The decimal number system is the system we use every day, employing the digits 0 through 9. It's called the 'decimal' system because it has a base, or radix, of 10. This base of 10 likely comes from humans having 10 fingers and toes.

In a positional number system like decimal, each digit's value depends on its position within the number. Each position has an associated weight, which is a power of the base (10 in this case).

Think of it like a special kind of odometer in a car. Each digit on the odometer represents a different power of 10. The rightmost digit represents units (10^0), the next to the left represents tens (10^1), then hundreds (10^2), and so on. Each position is worth 10 times the value of the position immediately to its right.

For example, the number 4728 can be broken down as:

- 4 is in the thousands position (4 * 10^3)
- 7 is in the hundreds position (7 * 10^2)
- 2 is in the tens position (2 * 10^1)
- 8 is in the units position (8 * 10^0)

This principle also applies to decimal fractions, but we use negative powers of 10. For instance, 0.256 means:

- 2 is in the tenths position (2 * 10^-1)
- 5 is in the hundredths position (5 * 10^-2)
- 6 is in the thousandths position (6 * 10^-3)

So, a number can have digits raised to both positive powers of 10 (for the whole number part) and negative powers of 10 (for the fractional part).

1.1.1 The Binary System

Digital circuits and systems rely entirely on the binary number system. This system has a base, also called a radix, of 2, meaning it only uses two digits: 0 and 1.

Similar to the decimal system we discussed, the binary system has an integer part (to the left of the binary point) and a fractional part (to the right of the binary point).

Each position in a binary number has a specific weight, which is a power of the base 2.

- To the left of the binary point, the positions represent weights of 2^0, 2^1, 2^2, 2^3, and so on, increasing as you move further left.
- To the right of the binary point, the positions represent weights of 2^-1, 2^-2, 2^-3, and so on, decreasing as you move further right.

Think of it like a light switch: it can only be in one of two states – on (1) or off (0). The binary system uses these two states to represent all information in a computer.

Example 1

We can convert a binary number to its equivalent decimal number using a mathematical formula. After performing the calculations, the result will be a decimal number.

Let's take the binary number 1101.011 as an example.

This number can be split into two parts:

- **Integer part:** 1101
- **Fractional part:** 0.011

Each digit in a binary number has a specific **weight**, which is a power of 2. These weights depend on the digit's position relative to the binary point (the equivalent of the decimal point).

For the integer part (1101):

- The rightmost digit '1' has a weight of 2^0.
- The next digit '0' has a weight of 2^1.
- The next digit '1' has a weight of 2^2.
- The leftmost digit '1' has a weight of 2^3.

For the fractional part (0.011):

- The digit immediately after the binary point '0' has a weight of 2^-1.
- The next digit '1' has a weight of 2^-2.
- The next digit '1' has a weight of 2^-3.

By multiplying each digit by its corresponding weight and summing the results, we obtain the decimal equivalent.

a) Decimal Number to other Bases Conversion

This section explains how to convert a decimal number (base 10) into a number in another base, often called base 'r'. This process is necessary when we need to represent numbers using different systems, like binary (base 2) or hexadecimal (base 16).

If a decimal number has both an integer part (the whole number part) and a fractional part (the part after the decimal point), we convert each part separately.

Converting the Integer Part:

To convert the integer part of a decimal number to another base 'r', we repeatedly divide the integer by 'r'.

1. **Divide:** Divide the integer part of the decimal number by the base 'r'.

2. **Note Remainder:** Record the remainder of this division.

3. **Use Quotient:** Use the quotient (the result of the division, ignoring the remainder) as the new number for the next step.

4. **Repeat:** Continue dividing the successive quotients by 'r' and noting the remainders until the quotient becomes zero.

5. **Order Remainders:** The remainders, when read in reverse order (from last to first), form the integer part of the equivalent number in base 'r'. The first remainder you noted is the least significant digit (the rightmost digit), and the last remainder is the most significant digit (the leftmost digit).

Analogy: Imagine you have a pile of coins (the decimal integer) and you want to repackage them into bags of a specific size (the new base 'r'). You keep filling bags until you can't fill a whole bag anymore. The coins left over are your remainder. You then take the filled bags and see how many you can form into larger bundles of a specific size (the next division). You continue this until you have no coins left. The number of coins in each bag and bundle, read in a specific order, tells you how many of each denomination you have in the new system.

Converting the Fractional Part:

To convert the fractional part of a decimal number to another base 'r', we use multiplication.

1. **Multiply:** Multiply the fractional part of the decimal number by the base 'r'.

2. **Note Carry:** Record the integer part (the whole number part before the decimal) of the result. This is often called the 'carry'.

3. **Use New Fraction:** Use the fractional part of the result as the new number for the next step.

4. **Repeat:** Continue multiplying the successive fractional parts by 'r' and noting the integer part (carry) until the fractional part becomes zero or you have obtained the desired number of equivalent digits.

5. **Order Carries:** The carries, when read in their normal sequence (from first to last), form the fractional part of the equivalent number in base 'r'. The first carry is the most significant digit of the fractional part (the digit immediately after the decimal point).

Analogy: Imagine you have a portion of a cake (the decimal fraction) and you want to divide it into smaller slices of a specific size (the new base 'r'). You cut off as many full slices as you can. The remaining piece of cake is the new fraction you work with. You repeat this process until you have only crumbs left or you've cut enough slices.

By applying these two methods to the integer and fractional parts individually, you can convert any decimal number into its equivalent representation in another base 'r'.

1.1.2 CONVERTING BETWEEN BINARY AND DECIMAL

This section focuses on the practical process of translating numbers between the binary (base-2) system, which computers use, and the decimal (base-10) system, which humans commonly use.

We've already learned how binary numbers are constructed using powers of 2. This knowledge is key to converting them to decimal.

Binary to Decimal Conversion:

To convert a binary number to its decimal equivalent, you multiply each binary digit (bit) by its corresponding positional weight (a power of 2) and then sum up these products.

- The rightmost bit has a positional weight of 2⁰ (which is 1).

• Moving left, the weights increase as powers of 2 (2¹, 2², 2³, and so on).

• For the fractional part of a binary number, the positional weights are negative powers of 2 (2⁻¹, 2⁻², 2⁻³, etc.), starting from the bit immediately to the right of the binary point.

Analogy: Think of each digit in a binary number as a light switch. If the switch is 'on' (a 1), it contributes its value (the power of 2) to the total. If it's 'off' (a 0), it contributes nothing. Summing up the contributions of all the 'on' switches gives you the decimal value.

Decimal to Binary Conversion:

We previously covered the systematic methods for converting decimal numbers to binary (or any other base 'r'). This involves:

• For the integer part: Repeatedly dividing the decimal number by 2 and recording the remainders. The binary number is formed by reading the remainders from bottom to top.

• For the fractional part: Repeatedly multiplying the fractional part by 2 and recording the integer part of the result. The binary fraction is formed by reading these integer parts from top to bottom.

Example 2

This example demonstrates the conversion of a decimal number with both an integer and a fractional part into its binary equivalent.

We'll convert the decimal number 58.25.

1. Converting the Integer Part (58):

To convert the integer part (58) to binary, we repeatedly divide by 2 and record the remainders.

• 58 divided by 2 is 29 with a remainder of 0. This is the Least Significant Bit (LSB).

• 29 divided by 2 is 14 with a remainder of 1.

• 14 divided by 2 is 7 with a remainder of 0.

• 7 divided by 2 is 3 with a remainder of 1.

• 3 divided by 2 is 1 with a remainder of 1.

• 1 divided by 2 is 0 with a remainder of 1. This is the Most Significant Bit (MSB).

Reading the remainders from bottom to top (MSB to LSB), the binary equivalent of 58 is 111010.

2. Converting the Fractional Part (0.25):

To convert the fractional part (0.25) to binary, we repeatedly multiply by 2 and record the integer part of the result.

• 0.25 multiplied by 2 is 0.5. The integer part is 0. This is the first bit after the binary point.

• We then take the fractional part, 0.5, and multiply it by 2: 0.5 multiplied by 2 is 1.0. The integer part is 1. This is the next bit.

Since the result is now 1.0, we stop.

The binary equivalent of the fractional part 0.25 is .01.

3. Combining the Parts:

Putting the integer and fractional parts together, the binary equivalent of the decimal number 58.25 is 111010.01.

b) Decimal to Binary Conversion

To convert a decimal number into its equivalent binary number, two main types of operations are performed:

1. **Integer Part Conversion**: The integer part of the decimal number is repeatedly divided by the base of the target system, which is 2 for binary. The remainders from each division, read from bottom to top, form the binary integer part.

    • Think of this like trying to make change for a large bill using only coins of a specific denomination. You keep taking out as many of the largest coin as possible (division) and the coins left over are your remainder.

2. **Fractional Part Conversion**: The fractional part of the decimal number is repeatedly multiplied by the base, which is 2 for binary. The integer part of each multiplication result becomes a binary digit, and the fractional part is carried over to the next multiplication. This process continues until the fractional part becomes zero or a desired level of precision is reached.

    • Imagine trying to measure a length that's less than a whole unit using only fractions of that unit. You repeatedly see how many of the smallest fraction fit into what's left and keep the leftover piece to measure with even smaller fractions.

Example 3

Let's work through an example to solidify our understanding of converting binary numbers to decimal.

Consider the binary number 1101.11.

To find its decimal equivalent, we apply the positional weights (powers of 2) to each digit:

1101.11■ = (1 *2³)* + *(1* 2²) + (0 *2¹)* + *(1* 2■) + (1 *2■¹)* + *(1* 2■²)

Calculating the powers of 2:

$2^3 = 8$

$2^2 = 4$

$2^1 = 2$

2■ = 1

$2■^1 = 0.5$

$2■^2 = 0.25$

Now, substitute these values back into the equation:

1101.11■ = (1 *8)* + *(1* 4) + (0 *2)* + *(1* 1) + (1 *0.5)* + *(1* 0.25)

1101.11■ = 8 + 4 + 0 + 1 + 0.5 + 0.25

1101.11■ = 13.75■■

Therefore, the decimal equivalent of the binary number 1101.11 is 13.75.

c) Binary Number to other Bases Conversion

Converting a binary number to a base other than decimal follows a different procedure than converting to decimal.

While we've learned how to convert binary to decimal by considering the positional weights (powers of 2), converting to other bases requires a distinct approach. This section will outline that specific method.

d) Binary to Hexa-Decimal Conversion

This section explains how to convert binary numbers to hexadecimal numbers. Both binary and hexadecimal are number systems, with binary having a base of 2 and hexadecimal having a base of 16.

The key relationship here is that four bits (binary digits) are equivalent to one hexadecimal digit, because 2 raised to the power of 4 (2^4) equals 16.

To perform this conversion, follow these two steps:

1. **Grouping Bits:** Starting from the binary point (which is like a decimal point but for binary numbers), group the binary digits (bits) into sets of four. You need to make these groups on both sides of the binary point.

    • If a group on either side doesn't have exactly four bits, add the necessary number of zeros to the extreme ends (left side for the integer part, right side for the fractional part) to complete the group of four. Think of it like padding a word with spaces at the beginning or end to make it fit a specific box.

2. **Converting Groups:** After grouping, convert each group of four bits into its corresponding hexadecimal digit. Each unique combination of four bits has a direct mapping to a single hexadecimal digit (0-9 and A-F).

For example, the binary group '1010' directly converts to the hexadecimal digit 'A'.

Binary to Decimal Conversion

To convert a binary number to its decimal equivalent, you'll follow a two-step process:

1. **Multiply bits by positional weights:** Each digit (bit) in the binary number is multiplied by its corresponding positional weight. Think of these weights like place values in our everyday decimal system, but for binary, they are powers of 2.

2. **Sum the products:** After multiplying each bit by its weight, you add up all the resulting products. This final sum is the decimal representation of the original binary number.

For example, if you have the binary number `1011`:

- The rightmost bit (1) is in the 2^0 position (which is 1).
- The next bit to the left (1) is in the 2^1 position (which is 2).
- The next bit (0) is in the 2^2 position (which is 4).
- The leftmost bit (1) is in the 2^3 position (which is 8).

So, the conversion would be (1 *8)* + *(0* 4) + (1 *2)* + *(1* 1) = 8 + 0 + 2 + 1 = 11. Therefore, binary `1011` is equal to decimal `11`.

Example 4

This example demonstrates converting a binary number with both an integer and fractional part into its hexadecimal equivalent.

Binary to Hexadecimal Conversion Steps:

1. **Grouping Bits:**
   - Divide the binary number into groups of 4 bits, starting from the binary point (the equivalent of the decimal point).
   - For groups that don't have 4 bits, pad them with zeros on the "extreme side" to make them 4 bits long. For the integer part, this means adding zeros to the left. For the fractional part, this means adding zeros to the right.
   - Example: The binary number `101110.01101` becomes `0010 1110.0110 1000` after padding.
2. **Converting Groups:**
   - Convert each group of 4 bits into its corresponding hexadecimal digit.
   - The hexadecimal digits are 0-9 and A-F, where A represents 10, B represents 11, C represents 12, D represents 13, E represents 14, and F represents 15.
   - Example: `0010` (binary) is 2 (hexadecimal), `1110` (binary) is `E` (hexadecimal), `0110` (binary) is 6 (hexadecimal), and `1000` (binary) is 8 (hexadecimal).

Therefore, the hexadecimal equivalent of `101110.01101` (binary) is `2E.68` (hexadecimal).

Hexadecimal to Binary Conversion:

The process of converting hexadecimal numbers back to binary is the reverse. Each hexadecimal digit is simply replaced by its 4-bit binary equivalent.

Example 5

This example demonstrates the conversion of a hexadecimal number to its binary equivalent.

To convert the hexadecimal number 65.4C to binary, we represent each hexadecimal digit with its 4-bit binary equivalent:

- '6' in hexadecimal is '0110' in binary.
- '5' in hexadecimal is '0101' in binary.
- '4' in hexadecimal is '0100' in binary.
- 'C' in hexadecimal is '1100' in binary.

Putting these together, we get:

65.4C■■ = 0110 0101 . 0100 1100 ■

Leading zeros in the integer part and trailing zeros in the fractional part of a binary number do not change its value. Therefore, we can simplify this to:

65.4C■■ = 1100101 . 010011 ■

This shows that the binary equivalent of the hexadecimal number 65.4C is 1100101.010011.

2.1 Instruction Codes

An instruction code is a group of bits that tells a computer to perform a specific operation. Think of it like a very precise command for the computer.

An instruction code needs to specify three main things:

- **The operation to be performed**: This is like the verb in a command, such as 'add', 'subtract', 'multiply', 'shift', or 'complement'. This part of the instruction code is called the **operation code** (or opcode).
- **Where to find the data (operands)**: The operation needs data to work on, and this data is typically stored in **registers**. Registers are small, fast storage locations within the computer.
- **Where to store the result**: After performing the operation, the result needs to be saved somewhere, usually in another register.

The **operation code** itself is a specific set of bits that defines the particular operation. The number of bits used for the operation code determines how many different operations the computer can perform. If a computer has 'n' bits for the operation code, it can support up to 2^n different operations.

2.0 Arithmetic Logical Unit

Instead of individual registers performing micro-operations (small, fundamental operations), the computer system uses a specialized unit called the Arithmetic Logical Unit (ALU).

The ALU is a crucial component within the CPU (Central Processing Unit) and is responsible for carrying out all logical and mathematical operations.

Here's how it works:

- **Input:** The contents of specific registers are fed into the ALU as input.

- **Processing:** The ALU performs the requested operation (either arithmetic like addition or subtraction, or logical like AND or OR).
- **Output:** The result of the operation is then transferred to a designated destination register.

Think of the ALU as the 'calculator' and 'decision-maker' of the computer. It takes numbers or data from different parts of the system (registers), performs calculations or comparisons, and sends the outcome back to where it's needed.

2.2.1 Common Bus System

In a basic computer system, which includes registers, a memory unit, and a control unit, it's essential to have ways to move data between these components, particularly between registers.

A **Common Bus System** is an efficient method for transferring data within a computer. Think of it like a shared highway system. Instead of each register having its own dedicated road to every other register, they all connect to this central highway. This reduces the number of connections needed.

To make this work, the outputs of the registers and the memory unit are all connected to this common bus. This allows data to be sent from any of these sources onto the bus, and then potentially to any destination that is connected to listen on the bus.

2.2 Stored Program Organisation

In a computer, instructions and data are kept in separate areas of memory.

The simplest computer designs use a processor register, often called an Accumulator (ACC), and instruction codes. Each instruction code has two parts: the first part tells the computer what operation to perform (like adding or moving data), and the second part specifies a memory address. This address indicates where the data (called an operand) needed for the operation can be found in memory. The operation is then carried out using the memory operand and whatever is currently stored in the Accumulator.

Load (LD

Before diving into instruction formats, it's helpful to understand how the system specifies where to find the data (operands) that an instruction will work with.

When an instruction needs to access data, the way it specifies that data is called the operand address.

There are a few ways an instruction can specify an operand:

- **Immediate Operand:** The operand itself is directly included as part of the instruction code. Think of it like a direct command: "Add 5 to this value," where "5" is the immediate operand.
- **Direct Address:** The instruction code specifies the memory address where the operand can be found. This is like saying, "Go to box number 10 in the warehouse and get the item from there."

Indirect Address: *The instruction code specifies a memory address that contains* another* memory address. This second memory address then points to the actual location of the operand. It's like saying, "Go to box number 10 in the warehouse. Inside that box, you'll find a note telling you which other box contains the item you need."

Data from the common bus can be loaded into registers. When the LD (Load) input of a specific register is enabled, that register will receive data from the bus during the next clock pulse. This is how data moves into registers for processing.

2. Register Reference Instruction

Register Reference Instructions are a specific type of instruction that tells the computer to perform an operation directly on its registers.

These instructions are identified by a unique code: the first three bits of the instruction must be '111' (which is the opcode), and the very first bit of the entire instruction must be a '0'.

The remaining 12 bits of the instruction are used to specify exactly *which* operation the computer should perform. Think of the '111' and the leading '0' as a special 'key' that unlocks the ability for the other 12 bits to command the registers.

1. Memory Reference Instruction

Memory reference instructions are a type of instruction that needs to access data stored in the computer's memory. To do this, these instructions must specify where in memory the data is located.

Memory reference instructions use a specific number of bits to identify the memory address. In this system:

- **12 bits** are used to specify the memory address.
- **1 bit** is used to indicate the addressing mode, which is often labeled as 'I'.

The 'I' bit determines how the address is interpreted:

- If **I = 0**, it signifies a **direct address**. This means the 12 bits directly point to the exact location in memory where the data is. Think of it like a direct street address – you know exactly where to go.

*If **I = 1**, it signifies an **indirect address**. In this case, the 12 bits point to a memory location that* contains* the actual address of the data. It's like being given a note with another address written on it – you have to go to the first address to find out where the data truly is.

3. Input-Output Instruction

Input-Output (I/O) instructions are a special type of instruction that allows the computer to interact with the outside world, such as receiving data from a keyboard or sending data to a screen.

These instructions are identified by a specific operation code, which is '111'. Additionally, the very first bit (the leftmost bit) of the instruction must be '1' to signal it's an I/O instruction.

The remaining 12 bits of the instruction are dedicated to specifying exactly which input or output operation needs to be performed. Think of these 12 bits as a detailed set of commands for the I/O system.

2.2.2 Computer Instructions

Computer instructions are the fundamental commands that tell a computer what to do. They are represented as sequences of bits.

The basic computer system uses three different formats for these instruction codes.

Each instruction code has two main parts:

- **Operation code (opcode):** This is a set of 3 bits that specifies the particular operation the computer should perform.
- **Remaining bits:** The other 13 bits of the instruction vary depending on the specific operation defined by the opcode. These bits often provide details about the data to be used or where to find it.

a) Immediate Mode

In immediate mode, the data the instruction needs to work with, called the **operand**, is directly included within the instruction itself.

Instead of having a field to specify a memory address where the data is stored, an immediate mode instruction has a field that *is* the operand.

Think of it like this: If you ask someone to "Add 7" to a number, you're not telling them *where* to find the 7 (like in a specific box in a cupboard). You're directly giving them the number 7 to use right away. The instruction "ADD 7" is an example, where "7" is the operand, meaning "Add the value 7 to the contents of the accumulator."

Format of Instruction

An instruction in a computer is like a command that tells the processor what to do. The 'format of an instruction' describes how this command is structured, essentially breaking it down into different parts.

Think of an instruction like a recipe. A recipe has different parts: what you need to do (e.g., 'chop'), the ingredients you need (e.g., 'onions'), and perhaps where to store the chopped onions (e.g., 'in a bowl'). Similarly, a computer instruction has fields that specify:

- **Operation Code (Opcode) Field:** This part tells the computer the specific operation to perform. It's like the verb in our recipe command.
- **Address Field:** This field specifies the location of the data the operation will work on. This could be a specific place in the computer's memory or a register (a small, fast storage location within the CPU). It's like telling the recipe where to find the 'onions'.

Mode Field: *This field tells the processor* how* to find the data specified by the address field. It determines the 'effective address', which is the actual location of the data. This is like specifying if the 'onions' are

already chopped or if they are whole and need to be chopped first.

Different computers can have instructions of varying lengths, meaning they might have more or fewer of these fields. The number of address fields often depends on how the computer's internal registers are organized.

2.2.3 Addressing Modes and Instruction Cycle

The **operation field** of an instruction tells the computer what task to perform. This operation will use data that is either in the computer's registers or in its main memory.

The **addressing mode** of an instruction determines how the data (or operand) needed for the operation is found. Think of it like a specific instruction for locating a treasure on a map.

Addressing modes are important for two main reasons:

- **Programming Versatility:** They give users more flexibility in how they write and structure their programs.
- **Efficiency:** They help reduce the number of bits needed in the instruction to specify where the data is located, making instructions more compact.

Types of Addressing Modes

This section will explore various ways a computer instruction can specify the data it needs to work with, known as addressing modes. These modes offer different strategies for locating the operand (the data itself) that an instruction will operate on.

We will examine each type of addressing mode individually to understand its unique approach to data access.

b) Register Mode

In Register Mode, the data (operand) that an instruction needs to work with is stored directly within a register inside the CPU (Central Processing Unit). The instruction itself doesn't contain the data, but rather the 'address' or identifier of the specific register where that data can be found.

Think of it like this: Instead of writing the actual phone number in a message, you'd write the name of a contact in your phone. The instruction is like the message, and the register name is like the contact name. The CPU knows where to find the actual 'phone number' (the data) associated with that contact (the register).

c) Register Indirect Mode

In Register Indirect Mode, the instruction doesn't contain the data (operand) itself, nor does it contain the direct memory address of the operand. Instead, the instruction points to a specific register within the CPU. The CPU then looks inside that register, and the value found there is the actual memory address where

the operand is located.

Think of it like this: Instead of giving you the exact location of a treasure chest (the operand), the instruction tells you to go to a specific landmark (the register). Once you reach that landmark, you'll find a map inside that tells you the treasure chest's actual location.

Key takeaway:

The register holds the address* of the operand, not the operand itself.

Advantages

Register Mode offers several benefits:

- **Faster memory access to the operand(s):** When the data an instruction needs to work with is directly in a CPU register, it can be accessed much more quickly than if it were stored in main memory. Think of it like having your frequently used tools right on your workbench (registers) instead of having to walk to a storage cabinet (memory) every time you need them.
- **Shorter instructions and faster instruction fetch:** Because a register is identified by a smaller number of bits compared to a full memory address, the instructions that use Register Mode can be shorter. This means fewer bits need to be transferred from memory to the CPU, leading to faster instruction fetching and execution.

Disadvantages

While using multiple registers can boost performance by allowing faster access to data, it introduces complexity. Specifically, it makes the instructions themselves more complicated.

Another limitation is a very limited address space. This means the computer can only access a restricted amount of memory.

d) Auto Increment/Decrement Mode

Auto Increment/Decrement Mode is an addressing mode where the value in a specified register is automatically adjusted after (post-increment/decrement) or before (pre-increment/decrement) its value is used to access data.

Think of it like using a shopping list. If you're using 'post-increment', you find the item at the current spot on your list, grab it, and *then* you move to the next item on the list for your next search. If you're using 'pre-increment', you first move to the next item on your list, and *then* you find and grab the item at that new spot.

e) `Direct Addressing Mode

Direct Addressing Mode is a way an instruction specifies where to find the data it needs to operate on. In this mode, the instruction itself contains the **effective address** of the operand. The effective address is

simply the actual location in memory where the data (the operand) is stored.

Think of it like this: if you want to mail a letter, and the instruction is "Go to mailbox number 4000 and get the letter inside," then "4000" is the direct address of the letter. You don't need to figure out any other steps to find it.

Key characteristics of Direct Addressing Mode:

- It requires only a single reference to memory to get the data.
- There are no extra calculations needed to figure out where the data is; the instruction tells you directly.

g) Displacement Addressing Mode

Displacement Addressing Mode is a way for an instruction to figure out where the data (operand) it needs is located in memory. It works by taking the 'Address part' of the instruction and adding it to the 'contents of the indexed register'. The result of this addition is the 'effective address' – the actual memory location where the operand can be found.

Think of it like a treasure map. The instruction has a general starting point (the Address part), and you have a special ruler (the indexed register) that tells you how much to move from that starting point. By combining the starting point and the ruler's measurement, you find the exact spot of the treasure (the operand).

- **Effective Address**: The final calculated memory address of the operand.
- **Indexed Register**: A CPU register whose contents are used to calculate the effective address.
- **Address Part**: The portion of the instruction that contains a base address or offset.

f) Indirect Addressing Mode

Indirect Addressing Mode is an addressing technique where the address field within an instruction doesn't directly point to the data itself. Instead, it provides the memory address where the *actual* memory address of the operand is stored.

Think of it like a treasure hunt:

- **Direct Addressing** is like being given the exact coordinates of the treasure.
- **Indirect Addressing** is like being given a map to a location, and at that location, you find another map with the treasure's coordinates.

This extra step of looking up the address in memory means that Indirect Addressing slows down the execution of instructions because it requires multiple trips to memory to find the final data.

h) Relative Addressing Mode

Relative Addressing Mode is a type of Displacement Addressing Mode.

In this mode, the **Effective Address (EA)**, which is the final memory address where the operand is located, is calculated by adding the contents of the **Program Counter (PC)** to the address part of the instruction.

The Program Counter (PC) is a special register that holds the memory address of the *next* instruction to be executed.

Essentially, the operand is located a certain number of "cells" away from the current instruction's location.

The formula is: EA = Address Part of Instruction + PC

Think of it like this: If you're reading a book and the current page (PC) tells you to look at a note on page 5 (address part of instruction) that's relative to where you are, you'd go to page 5 *from* your current page, not from the very beginning of the book. This helps in creating programs that can be easily moved around in memory.

j) Stack Addressing Mode

In Stack Addressing Mode, the operand is located at the top of the stack. The stack is a data structure that operates on a Last-In, First-Out (LIFO) principle, much like a stack of plates.

When an instruction like 'ADD' is used in this mode:

- It first removes (POPs) the top two items from the stack.
- It then performs the addition on these two items.
- Finally, it places (PUSHES) the result back onto the top of the stack.

i) Base Register Addressing Mode

Base Register Addressing Mode is a variation of Displacement Addressing Mode.

In this mode, the effective address (EA) of the operand is calculated by adding a displacement value (A) to the contents of a specific register (R), which holds a pointer to a base address. The formula is: EA = A + (R).

Think of it like this: Imagine you have a large filing cabinet (the computer's memory). The 'base address' (R) is like a label on a specific drawer where a section of related files is kept. The 'displacement' (A) is like a number on a specific file within that drawer. To find the exact file, you first go to the correct drawer (using R) and then count to the specific file (using A).

a) BCD Code or 8421 Code

BCD stands for 'Binary-Coded Decimal'. It's a way to represent decimal numbers using binary bits.

Think of it like using a special code where each decimal digit (0 through 9) gets its own unique 4-bit binary pattern. This is different from just converting the entire decimal number into its straight binary equivalent. For example, the decimal number 35 is represented in BCD as 0011 0101, not the straight binary 100011.

This code is also called the '8421 code' because each of the four bits has a specific weight: 8, 4, 2, and 1, from left to right (most significant bit to least significant bit).

To find the decimal value of a BCD code, you multiply each bit by its weight and add the results. For instance, the BCD code 0101 means:

(0 *8)* + *(1* 4) + (0 *2)* + *(1* 1) = 5.

Because BCD uses 4 bits, it can technically represent numbers up to 15 (binary 1111). However, in BCD, we only use the patterns that represent the decimal digits 0 through 9. This means the binary patterns 1010, 1011, 1100, 1101, 1110, and 1111 are not used. These unused patterns are called 'forbidden codes' or the 'forbidden group'.

While BCD often uses more bits than straight binary to represent a number, it's very convenient for input and output operations in digital systems because it keeps the decimal representation clear.

3.0 CODES AND THEIR CONVERSIONS

This section builds upon our understanding of number systems by exploring various codes used in computing and how to convert between them.

We've already seen how decimal and binary numbers work. Now, let's look at other ways information is represented.

BCD (Binary Coded Decimal) or 8421 Code:

- This code represents each decimal digit (0-9) using a 4-bit binary pattern.
- It's called '8421' because the bits in the 4-bit pattern correspond to weights of 8, 4, 2, and 1, respectively.

*For example, the decimal digit '5' is represented as '0101' (0*8 + 1*4 + 0*2 + 1*1 = 5).

- Importantly, BCD has specific 'forbidden' codes – combinations of 4 bits that do not represent any decimal digit (e.g., '1010', which is 10 in binary, is not a valid BCD representation for a single decimal digit).

Think of BCD like using a special 4-digit stamp for each number on a calculator's display, rather than a single continuous number line. Each stamp only accepts specific patterns.

Introductions

In digital systems, various codes are used for different tasks like entering data, performing calculations, and checking for errors. The choice of a specific code depends on what needs to be done.

Digital circuits work with binary signals (0s and 1s), but they often need to process information that isn't binary, such as letters, numbers, or special characters. To do this, any information that isn't already in binary format must be converted into a binary form that digital circuits can understand. This conversion process is called coding, where each character or numeral is represented by a unique combination of 0s and 1s.

Sometimes, a single digital system might use multiple types of codes for different operations. In such cases, it's important to be able to convert data from one code to another using special code converter circuits.

Codes can be broadly categorized into five main groups:

- **Weighted Binary Codes:** In these codes, each position in a number has a specific 'weight'. To find the decimal equivalent, you multiply each bit by its corresponding weight and then add up the results. (Example: BCD, which we've seen, is a type of weighted binary code).
- **Non-weighted Codes:** These codes don't assign a specific weight to each bit position.
- **Error-detection Codes:** These codes are designed to help identify if an error has occurred during data transmission or storage.
- **Error-correcting Codes:** These are more advanced codes that not only detect errors but can also correct them.
- **Alphanumeric Codes:** These codes represent a combination of alphabetic characters, numeric digits, and special symbols.

b) 84-2-1 Code

The 84-2-1 code is a type of decimal code that uses negative weights for its bits. Instead of just positive values like 8, 4, 2, and 1, it incorporates negative weights.

This code is notable for being **self-complementary**. This means that to find the 9's complement of a decimal number represented in 84-2-1 code, you simply flip all the bits (change 1s to 0s and 0s to 1s). This is the same as finding the 1's complement of the bit pattern.

For instance, the bit pattern 0101 represents the decimal digit 3. We can calculate this by summing the products of each bit and its corresponding weight: (0 *8)* + *(1* 4) + (0 *-2)* + *(1* -1) = 0 + 4 + 0 - 1 = 3.

If we flip the bits of 0101 to get 1010, this flipped pattern represents decimal 6. This is calculated as (1 *8)* + *(0* 4) + (1 *-2)* + *(0* -1) = 8 + 0 - 2 + 0 = 6. Since 6 is indeed the 9's complement of 3, this demonstrates the self-complementary nature of the 84-2-1 code.

Example 3.2

This example demonstrates how to convert a decimal number into its Binary Coded Decimal (BCD) equivalent.

Problem: Give the BCD equivalent for the decimal number 69.27.

Solution:

To find the BCD equivalent, we convert each decimal digit separately into its 4-bit BCD representation.

- The decimal number is 69.27.
- The integer part is 69.
- The digit '6' in decimal is represented as `0110` in BCD.

- The digit '9' in decimal is represented as `1001` in BCD.
- Combining these, the integer part `69` becomes `0110 1001` in BCD.
- The fractional part is .27.
- The digit '2' in decimal is represented as `0010` in BCD.
- The digit '7' in decimal is represented as `0111` in BCD.
- Combining these, the fractional part `.27` becomes `.0010 0111` in BCD.

Therefore, the decimal number (69.27)■■ is equivalent to the BCD code (01101001.00100111).

Key takeaway: BCD represents each decimal digit with a fixed 4-bit pattern, making it a straightforward way to encode decimal numbers for digital systems.

Example 3.1

This example demonstrates how to convert a decimal number into its Binary Coded Decimal (BCD) equivalent.

Problem: Give the BCD equivalent for the decimal number 589.

Solution:

1. **Identify the Decimal Number:** The decimal number to convert is 589.
2. **Convert Each Digit to BCD:** Each decimal digit is represented by a 4-bit binary pattern.
   - The decimal digit '5' is represented as 0101 in BCD.
   - The decimal digit '8' is represented as 1000 in BCD.
   - The decimal digit '9' is represented as 1001 in BCD.
3. **Combine the BCD Representations:** Concatenate the BCD codes for each digit in the same order as they appear in the decimal number.
   - 0101 (for 5) + 1000 (for 8) + 1001 (for 9) = 010110001001

Therefore, the BCD equivalent for the decimal number 589 is (010110001001)BCD.

3.1.2 Nonweighted Codes

Nonweighted codes are a type of code where each position in the binary representation is not assigned a fixed, positional value. Unlike weighted codes (like BCD or the binary system itself, where each bit's position has a specific power of 2 associated with it), in nonweighted codes, the position doesn't directly tell you its 'weight' in the same way.

Think of it like a secret handshake where the order of gestures matters, but each gesture doesn't have a numerical value assigned to it. The combination of gestures is what's important, not the individual 'value' of each gesture.

Examples of nonweighted codes include:

- **Excess-3 codes**: These codes are derived from BCD but have additional values added.

- **Gray codes**: These codes are known for having consecutive values that differ by only one bit.

c) 2421 Code

The 2421 code is another type of weighted code used to represent decimal digits. In this code, the weights assigned to the four bits are 2, 4, 2, and 1.

This code shares similarities with BCD for decimal digits 0 through 4. However, it differs for digits 5 through 9.

Like the 84-2-1 code, the 2421 code is also self-complementary. This means that to find the 9's complement of a decimal number represented in 2421 code, you simply invert all the bits (change 1s to 0s and 0s to 1s).

For example, the bit combination 0100 represents the decimal digit 4. To represent the decimal digit 7, the bit combination is 1101. We can verify this by calculating: (1 *2)* + *(1* 4) + (0 *2)* + *(1* 1) = 2 + 4 + 0 + 1 = 7.

a) Excess-3 Code

The Excess-3 code is a nonweighted code that was used in some older computers. It represents decimal digits using 4-bit binary patterns.

To get the Excess-3 code for a decimal digit, you first add 3 to that digit, and then find the 4-bit binary representation of the result. For example, the maximum decimal digit is 9. Adding 3 gives us 12. The 4-bit binary equivalent of 12 is 1100.

Like the 84-2-1 and 2421 codes, Excess-3 is also a self-complementary code. This means that if you flip all the bits (change 1s to 0s and 0s to 1s) of an Excess-3 code, you get the Excess-3 code for the 9's complement of the original decimal digit. This self-complementary property is very useful for performing subtraction in digital systems.

Example 3.4

This example demonstrates how to convert the decimal number (58.43)10 into its Excess-3 code.

To achieve this, we add 3 to each decimal digit before converting it to its 4-bit binary equivalent.

1. **Decimal Number:** 58.43
2. **Add 3 to each digit:**
    - 5 + 3 = 8
    - 8 + 3 = 11
    - 4 + 3 = 7
    - 3 + 3 = 6
3. **Convert each sum to its 4-bit binary equivalent:**
    - 8 becomes 1000

- 11 becomes 1011
- 7 becomes 0111
- 6 becomes 0110

Combining these binary equivalents gives us the Excess-3 code for (58.43)10.

Hence, the Excess-3 code for (58.43)10 = (1000 1011.0111 0110).

Example 3.3

This example demonstrates converting a decimal number to its Excess-3 code.

The process involves adding 3 to each individual decimal digit of the number before converting that sum into its 4-bit binary equivalent.

Let's convert the decimal number (367)10:

1. Take the first digit, 3. Add 3: 3 + 3 = 6. Convert 6 to its 4-bit binary: 0110.

2. Take the second digit, 6. Add 3: 6 + 3 = 9. Convert 9 to its 4-bit binary: 1001.

3. Take the third digit, 7. Add 3: 7 + 3 = 10. Convert 10 to its 4-bit binary: 1010.

Combining these 4-bit binary equivalents gives us the Excess-3 code for (367)10, which is 0110 1001 1010.

Example 3.5. Convert (101011)2 Solution

This example demonstrates the conversion of a binary number (101011) into its Gray code equivalent. The process involves a series of bitwise operations:

- **Step 1: The Most Significant Bit (MSB)** The first bit of the Gray code is identical to the MSB of the original binary number. For (101011)2, the MSB is 1, so the first bit of the Gray code is also 1.
- **Step 2: Second Bit** The second bit of the Gray code is found by performing an exclusive OR (ex-OR) operation between the MSB and the second bit of the binary number. In (101011)2, this is ex-OR(1, 0), which results in 1. This is the second bit of the Gray code.
- **Step 3: Third Bit** The third bit is obtained by performing an ex-OR operation between the second and third bits of the binary number: ex-OR(0, 1), which results in 1. This is the third bit of the Gray code.
- **Step 4: Fourth Bit** The fourth bit is the result of an ex-OR operation between the third and fourth bits of the binary number: ex-OR(1, 0), which results in 1. This is the fourth bit of the Gray code.
- **Subsequent Bits** This pattern continues for all bits. For example, the fifth bit would be ex-OR(0, 1) = 1, and the sixth bit would be ex-OR(1, 1) = 0.

Following these steps, the binary number (101011)2 is converted to its Gray code representation, which is (111110).

Binary to Gray Code Conversion Steps

Converting a binary number to its equivalent Gray code involves a straightforward, sequential process:

1. **MSB is the same:** The most significant bit (MSB) of the Gray code is identical to the MSB of the original binary number.

2. **Subsequent bits using XOR:** For every subsequent bit in the Gray code, you perform an exclusive OR (Ex-OR) operation on two adjacent bits of the binary number:

- The second Gray code bit is the Ex-OR of the first and second binary bits.
- The third Gray code bit is the Ex-OR of the second and third binary bits.
- This pattern continues for all remaining bits.

What is Exclusive OR (Ex-OR)?

The Ex-OR operation results in a '1' if the two input bits are different, and a '0' if they are the same. Think of it like a 'difference detector': it signals when things change.

Analogy: Imagine you're tracking changes in a sequence of lights. The first light is on (MSB). For each subsequent light, you note whether it's different from the one before it. If the current light is the same as the previous one, you mark a '0'; if it's different, you mark a '1'. This sequence of marks is your Gray code.

b) Gray Code

Gray code is a type of code known as a 'minimum change code'. This means that as you move from one number to the next in sequence, only a single bit changes.

Think of it like a special kind of odometer where instead of all the digits changing, only one ever flips at a time. For example, going from 001 to 011 is a change of only one bit (the middle one).

This property makes Gray code very useful in certain applications:

- **Shaft encoders:** Devices that measure rotational position.
- **Analog-to-digital converters (ADCs):** Used in many electronic devices to convert real-world analog signals into digital information.

It's important to note that Gray code is not a weighted code. This means that, unlike BCD where each bit position has a fixed value (like 8, 4, 2, 1), the position of a bit in Gray code doesn't have a specific numerical weight assigned to it.

Gray to Binary Conversion Algorithm

This section explains the step-by-step process for converting a Gray code number into its equivalent binary number.

The algorithm involves the following steps:

- **Step 1: The Most Significant Bit (MSB)**

The MSB of the binary number is identical to the MSB of the Gray code.

- **Step 2: Subsequent Bits**

Each subsequent bit in the binary number is determined by taking the Exclusive OR (Ex-OR) of the *previous* binary bit and the *current* Gray code bit.

- The Ex-OR operation results in 0 if the two bits are the same, and 1 if they are different.
- For example, the second binary bit is the Ex-OR of the first binary bit (which is the MSB) and the second Gray code bit.
- The third binary bit is the Ex-OR of the second binary bit and the third Gray code bit, and so on for all remaining bits.

Example 3.6. Convert (564)10 to Gray code Solution

This example demonstrates how to convert a decimal number, (564)10, into its Gray code equivalent.

Step 1. Convert the decimal number 564 into its equivalent binary representation.

The decimal number 564 is converted to its binary form, which is 1000110100.

(Note: The conversion of decimal to binary was previously covered, involving repeated division by 2 for the integer part.)

Example 3.7. Convert the Gray code 101101 into a binary number

This example demonstrates how to convert a Gray code number to its binary equivalent. The process relies on specific bitwise operations.

Steps for Gray to Binary Conversion:

1. **MSB Match:** The Most Significant Bit (MSB) of the binary number is identical to the MSB of the Gray code number. For the Gray code 101101, the MSB of the binary number is 1.

2. **Ex-OR for Next Bit:** To find the next bit of the binary number, perform an Exclusive OR (Ex-OR) operation between the previously determined binary bit and the corresponding bit in the Gray code. The result of this operation becomes the next binary bit.

- For the second binary bit: Ex-OR the first binary bit (1) with the second Gray code bit (0). 1 Ex-OR 0 = 1. So, the second binary bit is 1.

3. **Repeat Ex-OR:** Continue this process for each subsequent bit. For each new binary bit, perform an Ex-OR operation between the *previous* binary bit and the *current* Gray code bit.

- For the third binary bit: Ex-OR the second binary bit (1) with the third Gray code bit (1). 1 Ex-OR 1 = 0. So, the third binary bit is 0.
- For the fourth binary bit: Ex-OR the third binary bit (0) with the fourth Gray code bit (1). 0 Ex-OR 1 = 1. So, the fourth binary bit is 1.
- For the fifth binary bit: Ex-OR the fourth binary bit (1) with the fifth Gray code bit (0). 1 Ex-OR 0 = 1. So, the fifth binary bit is 1.
- For the sixth binary bit: Ex-OR the fifth binary bit (1) with the sixth Gray code bit (1). 1 Ex-OR 1 = 0. So, the sixth binary bit is 0.

Following these steps, the Gray code 101101 converts to the binary number 110110.

a) Parity Bit Coding Technique

Binary information can be transmitted through various communication media like wires, radio waves, or fiber optic cables. However, external noise can interfere with this transmission, causing bit values to flip (e.g., 0 to 1 or 1 to 0). This makes binary data transmission susceptible to errors.

A **parity bit** is an extra bit added to a message to ensure that the total count of '1's in the message, including the parity bit, is either always odd or always even. This is a type of **error-detection code**, meaning it can identify if an error has occurred during transmission, but it cannot fix the error.

At the sending end, a message (e.g., the first four bits) is fed into a **parity generation** circuit, which calculates and adds the necessary parity bit (P). The combined message and parity bit are then sent.

At the receiving end, a **parity check** circuit receives all incoming bits. It checks if the parity of the received bits matches the expected parity (either odd or even). If the checked parity doesn't match the adopted parity rule, an error is detected.

How it works:

- **Odd Parity:** The parity bit is chosen so that the total number of 1s in the message (including the parity bit) is odd.
- **Even Parity:** The parity bit is chosen so that the total number of 1s in the message (including the parity bit) is even.

The parity bit can be placed either as the Most Significant Bit (MSB) or the Least Significant Bit (LSB) of the data.

Limitations:

The parity method can detect errors involving an odd number of bit flips (one, three, five, etc.). However, an even number of errors (like two bit flips) will not change the total count of 1s, making them undetectable by this method. For instance, if you have even parity and two bits flip, the count of 1s remains even, and the error goes unnoticed.

3.1.3 Error-detection Codes

Error-detection codes are a crucial part of digital systems, designed to identify when mistakes have occurred during data transmission or storage. The primary goal is to ensure the integrity of the data.

One common method is the **parity bit** technique. This involves adding an extra bit to a block of binary data. The value of this parity bit is set to make the total number of '1's in the block either always odd or always even, depending on the chosen parity scheme.

Imagine sending a secret message where each word has a specific number of letters. The parity bit is like a quick check at the end of the message to see if the letter count is correct. If the count is off, you know something went wrong during the transmission.

- **Parity Bit**: An extra bit added to binary data.
- **Purpose**: To detect transmission or storage errors.
- **Odd Parity**: The total number of 1s (including the parity bit) is odd.
- **Even Parity**: The total number of 1s (including the parity bit) is even.

b) Check Sums

Check Sums are a method used to detect errors in transmitted data, particularly useful when the parity bit technique might fail (e.g., with double errors).

Here's how it works:

1. **Accumulating the Sum:** At the transmitting end, the binary digits of each transmitted word are added together. This sum is then retained. As each new word is sent, it's added to the previously retained sum.

2. **Transmitting the Check Sum:** After all words have been transmitted and added, the final sum is calculated. This final sum is then also transmitted along with the data.

3. **Verification at the Receiver:** The receiving end performs the exact same addition operation on the received words. It calculates its own final sum.

4. **Error Check:** The receiver then compares its calculated sum with the transmitted Check Sum. If the two sums are equal, it indicates that there was no error during transmission.

Think of it like this: Imagine you're sending a list of numbers to a friend. Before sending, you add them all up and write down the total (your 'Check Sum'). You then send the list of numbers and the total. Your friend receives the list, adds them up on their end, and compares their total to the one you sent. If they match, they know they received all the numbers correctly.

a) Hamming Code

The Hamming Code is a method developed by R. W. Hamming for detecting and correcting errors in binary data. It works by systematically adding one or more parity bits to a data character.

Key concepts:

- **Hamming Distance (dij):** Defined as the number of bit positions in which two code words differ. The minimum Hamming distance (dmin) across all pairs of code words in a block code is a crucial property.
- A greater dmin provides better error detection and correction capabilities.
- For single error detection, dmin should be at least two.
- For single error correction, dmin should be at least three, as the number of correctable errors E < (dmin - 1)/2.

Encoding a Hamming Code (e.g., 7-bit Hamming (7, 4) code for 4 data bits b3b2b1b0):

- Parity bits (h1, h2, h4, ...) are placed in positions corresponding to powers of two (1, 2, 4, 8, ...).
- Data bits fill the remaining positions.
- Parity bits are calculated to ensure a specific parity (e.g., even parity) across certain groups of bits.
- For example, h1 checks bits b1, b2, b3, ...

- h2 checks bits b2, b3, b4, ...
- h4 checks bits b4, b5, b6, ...

Decoding and Error Correction:

- To detect errors, a check is performed, often for odd parity on the same bit fields used for even parity during encoding.
- A non-zero parity word (e.g., a3a2a1) indicates an error.
- The value of the parity word directly indicates the position of the erroneous bit. For instance, if the parity word is 110 (binary), bit 6 is in error.
- To correct a single-bit error, the bit at the identified error position is complemented (flipped).
- If the parity word is 000, it signifies that no error was detected.

Example of Error Detection and Correction:

Received code: 0110110

- The parity check might result in a parity word of 010.
- This 010 (binary) indicates that bit 2 is in error.
- Complementing bit 2 of the received code (0110110) yields the corrected code: 0010110.

3.1.4 Error-correcting Codes

While error-detection codes, like parity bits and checksums, can tell us if an error has occurred, they can't always fix it. Error-correcting codes go a step further by not only detecting errors but also identifying and correcting them.

The Hamming Code is a prime example of an error-correcting technique. It works by strategically adding extra 'parity bits' to the original data. These parity bits are calculated based on specific subsets of the data bits.

When data is transmitted, and an error occurs, the receiver recalculates the parity bits based on the received data. By comparing these recalculated parity bits with the received parity bits, the Hamming Code can pinpoint the exact location of a single-bit error. Once the erroneous bit is identified, it can be flipped, thereby correcting the error.

Think of it like a detective who not only finds out a mistake was made in a report but also figures out exactly which word was wrong and corrects it. The Hamming Code uses a system of 'syndrome bits' (derived from comparing expected and actual parity) to act as the detective's clue, pointing directly to the faulty bit.

3.1.5 Alphanumeric Codes

Alphanumeric codes are essential for computers to handle information beyond just numbers. They provide a way to represent letters of the alphabet, decimal digits, and various special characters (like #, /, &, %) using binary patterns.

Because alphanumeric codes need to represent more than just the 36 elements (26 letters + 10 digits), they require a minimum of 6 bits. This is because 5 bits can only represent 2^5 = 32 unique patterns, which isn't enough, while 6 bits can represent 2^6 = 64 unique patterns, providing sufficient room.

Think of it like a special secret code. If you only need to send messages about numbers 0-9, a short code (like BCD, which uses 4 bits) is fine. But if you also want to include all the letters (A-Z), plus punctuation, you need a longer code to make sure every character has its own unique binary representation.

3.1.7 EBCDIC

EBCDIC stands for "Extended Binary Coded Decimal Interchange Code." It's an 8-bit alphanumeric code, meaning it uses 8 bits to represent letters, numbers, and special characters. An additional ninth bit is often added for parity checking, a method used for error detection.

EBCDIC is commonly used in IBM equipment and larger computer systems for handling alphanumeric data. Its grouping of characters differs from other alphanumeric codes like ASCII.

3.1.6 ASCII

ASCII, which stands for "American Standard Code for Information Interchange" (and is pronounced "as-kee"), is a widely used code in most microcomputers.

It's a 7-bit code, meaning each character is represented using seven bits. When stored, a character typically takes up one byte (8 bits), with one bit left unused. This extra bit is often utilized to expand the ASCII code, allowing for an additional 128 characters to be represented.

Think of it like assigning a unique, short numerical ID to each letter, number, and symbol. ASCII is like a common language for computers to agree on what that ID is for, say, the letter 'A' or the number '5'. The 7 bits are like the digits in that ID, and the extra bit can be used to expand the library of available IDs.

4.0 BOOLEAN ALGEBRA

Boolean algebra is a mathematical system used for designing and analyzing digital circuitry. It was first suggested by Claude Shannon in 1938 for solving problems in relay-switching circuit design and is named after George Boole, who proposed its principles in 1854.

Boolean algebra has two main uses:

- **Analysis:** It provides an economical way to describe the function of digital circuitry.
- **Design:** It helps develop simplified implementations of desired functions.

Key concepts in Boolean algebra include:

- **Logical Variables:** These variables can take on one of two values: 1 (TRUE) or 0 (FALSE).
- **Basic Logical Operations:** The fundamental operations are AND, OR, and NOT. These are often represented symbolically by a dot (.), a plus sign (+), and an overbar (‾) respectively.
- **Other Operations:** Additional operations include NOR, NAND, and XOR.

A **truth table** is a table that shows the output for all possible combinations of inputs to a logic gate or circuit. Truth tables can be constructed for Boolean operators, starting with two input variables and extending to more.

For example, the AND operation results in 1 only if all inputs are 1, while the OR operation results in 1 if at least one input is 1. The NOT operation inverts the input (0 becomes 1, and 1 becomes 0).

4.1 Gates

The fundamental building blocks of all digital logic circuits are called **gates**. These are electronic circuits that take input signals and produce an output signal based on a simple Boolean operation. Think of them like tiny decision-makers: based on the inputs they receive, they decide whether to output a '1' (representing true or high voltage) or a '0' (representing false or low voltage).

There are six basic types of gates used in digital logic:

• **AND Gate:** An AND gate outputs a '1' only if **all** of its inputs are '1'. If even one input is '0', the output is '0'. It's like a series of light switches wired so the light only turns on if every single switch is flipped up.
• Algebraic notation: F = A · B (or AB)
• Truth Table: Shows all possible input combinations and their corresponding output.
• Graphical Symbol: Represented by a D-shaped symbol.

• **OR Gate:** An OR gate outputs a '1' if **any** of its inputs are '1'. It only outputs '0' if all its inputs are '0'. This is like having multiple buttons that can turn on a light – pressing any one of them will do the job.
• Algebraic notation: F = A + B
• Truth Table: Shows all possible input combinations and their corresponding output.
• Graphical Symbol: Represented by a curved, pointed symbol.

• **NOT Gate (Inverter):** A NOT gate has only one input and outputs the opposite of its input. If the input is '1', the output is '0', and vice-versa. It's like a simple light switch that turns the light ON when you flip it DOWN and OFF when you flip it UP.
• Algebraic notation: F = A'
• Truth Table: Shows the single input and its inverted output.
• Graphical Symbol: Represented by a triangle with a small circle at the output.

• **NAND Gate:** A NAND gate is essentially an AND gate followed by a NOT gate. It outputs '0' only if all of its inputs are '1'. For all other input combinations, it outputs '1'.
• Algebraic notation: F = (A · B)'
• Graphical Symbol: Similar to an AND gate symbol but with a small circle at the output.

• **NOR Gate:** A NOR gate is an OR gate followed by a NOT gate. It outputs '1' only if all of its inputs are '0'. For all other input combinations, it outputs '0'.
• Algebraic notation: F = (A + B)'
• Graphical Symbol: Similar to an OR gate symbol but with a small circle at the output.

• **XOR Gate (Exclusive OR):** An XOR gate outputs '1' if the inputs are different (one is '0' and the other is '1'). If the inputs are the same (both '0' or both '1'), it outputs '0'. This is like a light switch that only

toggles the light ON if you press a different button than the one you pressed last.

- Algebraic notation: $F = A \oplus B$
- Truth Table: Shows all possible input combinations and their corresponding output.
- Graphical Symbol: Similar to an OR gate symbol but with an extra curved line at the input.

Each of these gates is defined by its graphical symbol, its algebraic notation (how it's represented mathematically), and its truth table (which lists all possible input combinations and their resulting outputs). These gates are interconnected to implement complex logical functions.